

Tarea No. 06

María Fernanda Orellana Morales, 202603539
Escuela de Mecánica Eléctrica, Facultad de Ingeniería,
Universidad de San Carlos de Guatemala

Resumen—Un puntero en lenguaje C es una variable encargada de almacenar la dirección de memoria de otra variable. Para su correcto funcionamiento, resulta indispensable inicializarlos antes de aplicarles el operador de desreferenciación *, pues de lo contrario se generan punteros salvajes que provocan fallos críticos en la ejecución, tales como el error de segmentación (*Segmentation fault*). Asimismo, la aritmética de punteros posibilita el desplazamiento secuencial a través de las ubicaciones de memoria. Dicha funcionalidad facilita la manipulación eficiente de arreglos y cadenas de texto mediante el recorrido de caracteres o el empleo de matrices de punteros. Aunque representan herramientas de gran utilidad para la optimización de recursos y el acceso directo a la memoria, su manejo demanda rigurosidad para prevenir vulnerabilidades y fallas operativas.

I. MARCO TEÓRICO

E

El aprendizaje del uso del lenguaje de programación C requiere que se posean un conjunto de conocimientos básicos sobre los recursos del sistema. Esta exposición contiene 4 conceptos que son fundamentales para comprender y manejar adecuadamente las técnicas de manejo de la memoria en el desarrollo e instalación de programas.

A. Puntero

Variable cuya función es almacenar la dirección de otra variable, esto reduce el uso de los recursos y permite el acceso rápido a la información.

B. Desreferenciación

Consiste en realizar una consulta o modificar el contenido del área de la memoria que el puntero indicó usando asterisco (*) como operador.

C. Aritmética de Punteros

El puntero se desplaza al realizar una operación, asignar, sumar, restar, decrementar o incrementar el puntero según sea el tamaño en bytes de su tipo básico.

D. Puntero Salvaje

Esta situación representa que el puntero no fue dado un valor con el cual poder trabajar, se puede mover a cualquier dirección en la que apunte e incluso ocasionar fallos graves que terminen el programa abruptamente, como la violación de segmento.

II. OBJETIVOS

Elaborar un sistema en C de gestión y análisis de punteros que permita entender, aplicar y depurar eficazmente conceptos avanzados de manejo de memoria, optimizando el rendimiento y evitando fallos críticos de ejecución como errores de segmentación.

A. Específicos

- Conocer el funcionamiento de los punteros y sus tipos mediante la realización de al menos 5 casos prácticos en entornos de desarrollo basados en Linux.
- Emplear la desreferenciación y la aritmética de punteros para el recorrido eficiente de arreglos y matrices de cadenas de caracteres.
- Localizar las causas de los errores de segmentación por punteros no inicializados o punteros salvajes y corregir la sintaxis del código fuente.
- Recopilar los conceptos teóricos y prácticos imprescindibles de la gestión de memoria para su elaboración en trabajos académicos.

III. MARCO PRÁCTICO

En el bloque de código práctico se lleva a cabo la declaración de punteros empleando tipos de datos base y el operador asterisco (`int *y = &v;`). También se ilustra la asignación de direcciones de memoria con el operador “ampersand” (`pointer = &variable`) se asegura la correcta inicialización de variables tipo entero, doble, flotante y carácter para evitar fallos de ejecución.

```
int *y = &v;
```

Fig. 1. Almacenamiento de la Dirección de una Variable.

```
data_type *pointer_variable_name;
```

Fig. 2. Declaración de un Puntero.

```
int    *ptr_thing;    /* pointer to an integer */
int    *ptr1, thing;  /* ptr1 points to an int; thing is an int */
double *ptr2;         /* pointer to a double */
float  *ptr3;         /* pointer to a float */
char   *ch1;          /* pointer to a character */
```

Fig. 3. Tipos de Datos.

```
pointer = &variable;
```

Fig. 4. Inicialización de un Puntero.

^{1*} Revista Argentina de Trabajos Estudiantiles. Patrocinada por la IEEE.

IV. CÓDIGO

```

GNU nano 8.7.1                                tarea06a.c
#include <stdio.h>
int main ()
//Inicializar un puntero.

{
    int a = 10; //Declaracion de variable.
    int *p; //Declaracion de variable del puntero.
    p = &a; //almacenar direccion del puntero p.
    printf("\n");
    printf("La direccion almacenada en p es: %p\n", (void*)p); //Acceso a la direccion.
    printf("\n");
    printf("Valor almacenado en p es: %d\n", *p); //Acceso al valor.
    return 0;
}

```

Fig. 1.Muestra cómo un puntero puede almacenar la dirección de una variable entera e imprime tanto el puntero como el valor de la variable a la que apunta.

```

GNU nano 8.7.1
#include <stdio.h>
int main ()
//Puntero Nulo
{
    int *p = NULL; //puntero nulo
    printf("\n");
    printf("El valor dentro de la variable p es: %p", p);
    printf("\n");
    return 0;
}

```

Fig. 2.Utiliza un puntero nulo ('NULL') que no apunta a ninguna dirección de memoria válida.

```

GNU nano 8.7.1                                tarea06c.c *
#include <stdio.h>
int main ()
{
    void *p = NULL; //Puntero Vacio
    printf("\n");
    printf("El tamaño del puntero es: %zu\n", sizeof(p)); //size of: valor de long unsigned int, 64 bit
    printf("\n"); //%d solo acepta enteros normales. %zu, para size_t.
    return 0;
}

```

Fig. 3.Cobra un puntero genérico ('void*') y calcula el número de bytes que ocupa dicho puntero con el operador 'sizeof'.

```

GNU nano 8.7.1                                tarea06d.c *
#include <stdio.h>
int main()
{
    int a = 10;
    int *p = &a;
    printf("\n");
    printf("%d", *p);
    printf("\n");
    return 0;
}
//Error inicial: violacion de segmento. Int *p puntero no inicializado. C es puntero salvaje.
//p no apunta ninguna direccion. Dicho esto, hay que asignar una variable antes de imprimir.

```

Fig. 4.Lleva a cabo la desreferenciación de un puntero para imprimir directamente el valor al que apunta.

```

GNU nano 8.7.1
#include <stdio.h>
//Puntero de Acceso Directo e Indirecto
int var = 1; //Declarar e iniciar la variable.
int *ptr; //Declarar un puntero a INT.
int main(void)
{
    ptr = &var; //Puntero ptr hacia var.
    printf("\n");
    printf("Acceso Indirecto, var = %d", var);
    printf("\n");
    printf("Acceso Indirecto, var = %d", var);
    printf("\n");
    *ptr = 48; //Cambiar a var a través del puntero.
    printf("Acceso Indirecto, var = %d", *ptr);
    printf("\n");
    return 0;
}

```

Fig. 5.Explica cómo podemos acceder a la variable tanto directamente como indirectamente a través de punteros consultar y modificar su valor.

```

GNU nano 8.7.1
#include <stdio.h>
int main()
//Punteros y Arreglos en C
{
    int a[5] = {1, 2, 3, 4, 5}; //Iniciar array
    int *p; //Declaracion de puntero
    p = a; //P se direcciona al primer elemento; igual a &a[0]
    for (int i = 0; i < 5; i++)
    {
        printf("\n");
        printf("%d\n", *p); //Imprimir elemento
        p++; //Moverse al siguiente elemento
    }
    return 0;
}

```

Fig. 6.Recorre un arreglo de números enteros mediante aritmética de punteros para avanzar de un elemento al siguiente.

```

GNU nano 8.7.1
#include <stdio.h>
#include <string.h>
//Punteros C y Cadenas
int main ()
{
    char str[] = "Hola Guru99!";
    char *p;

    p = str; //Inicializar el puntero direccionado a la cadena
    p = p + 1;

    printf("\n");
    printf("El siguiente caracter es: %c\n", *p);
    printf("\n");

    p = str; //Resetea el puntero
    for (int i = 0; i < strlen(str); i++)
    {
        printf("\n");
        printf("%c\n", *p);
        printf("\n");
        p++;
    }
    return 0;
}

```

Fig. 7.Trabaja con un puntero a caracteres para imprimir caracteres específicos de una cadena, así como recorrer la cadena completa con un bucle.

```
#include <stdio.h>
int main()
//Puntero con Cadenas Mediante Una Matriz de Punteros
{
    char *materiales[] = {"metal", "cobre", "oro"};
    for (int i = 0; i < 3; i++)
    {
        printf("\n");
        printf("%s\n", materiales[i]);
        printf("\n");
    }
    return 0;
}
```

Fig. 8. Emplea una matriz de punteros (char *materiales[]) para almacenar y mostrar de forma secuencial una lista de palabras.

V. RESULTADOS

```
La direccion almacenada en p es: 0x7ffde6f1f8cc
Valor almacenado en p es: 10
maferom@ubuntulinux:~/0769$ _
```

Fig. 9. Resultado de Fig. 1.

```
maferom@ubuntulinux:~/0769$ ./tarea06b
El valor dentro de la variable p es: (nil)
maferom@ubuntulinux:~/0769$ _
```

Fig. 10. Resultado de Fig. 2.

```
El tamaño del puntero es: 8
maferom@ubuntulinux:~/0769$ _
```

Fig. 11. Resultado de Fig. 3.

```
maferom@ubuntulinux:~/0769$ ./tarea06d
10
maferom@ubuntulinux:~/0769$ _
```

Fig. 12. Resultado de Fig. 4.

```
maferom@ubuntulinux:~/0769$ ./tarea06e
Acceso Indirecto, var = 1
Acceso Indirecto, var = 1
Acceso Indirecto, var = 48
maferom@ubuntulinux:~/0769$
```

Fig. 13. Resultado de Fig. 5.

```
maferom@ubuntulinux:~/0769$ ./tarea06f
1
2
3
4
5
maferom@ubuntulinux:~/0769$
```

Fig. 14. Resultado de Fig. 6.

```
El siguiente caracter es: o
H
o
1
a
G
u
r
u
9
9
!
maferom@ubuntulinux:~/0769$ _
```

Fig. 15. Resultado de Fig. 7.

```
maferom@ubuntulinux:~/0769$ ./tarea06h

metal

cobre

oro

maferom@ubuntulinux:~/0769$ _
```

Fig. 16. Resultado de Fig. 8.

VI. CONCLUSIONES

Elaborar esta práctica ha sido un complemento para afianzar tanto los conocimientos teóricos como los prácticos relacionados con el uso de punteros en lenguaje C bajo el entorno de *Ubuntu*. A través de la implementación y depuración de diversos códigos se ha comprendido la importancia de que las variables de memoria estén correctamente inicializadas, ya que de no ser así, pueden presentarse errores significativos, entre los cuales son muy comunes los *punteros salvajes* y fallos de segmentación (*Segmentation fault*). Además, el hecho de haber analizado ejemplos relacionados con la aritmética de punteros, el recorrido de arreglos o la manipulación de cadenas, ha dejado claro que el empleo de estas herramientas permite optimizar el acceso directo a la memoria y, por ende, maximizar la eficiencia del software.

REFERENCIAS

- [1] B. Walker, "Punteros en C: ¿Qué es la programación con punteros en C? Tipos," Guru99, Jun. 30, 2026. [En línea]. Disponible: <https://www.guru99.com/es/c-pointers.html> (consultado el Sep. 4, 2026).
- [2] "IEEE Referencing Style Guide," Scribd, 2024. [En línea]. Disponible: <https://www.scribd.com/presentation/823088728/IEEE-Method-of-Referencing> (consultado el Sep. 4, 2026).